

FraudGuard AI — Manual Explicativo del Código

Documentación Técnica Exhaustiva y Función de Cada Archivo del Proyecto
Conexión PostgreSQL Verificada | Inferencia ML | Explicabilidad | Frontend React

1. Introducción y Arquitectura del Sistema

El proyecto **FraudGuard AI** es una solución web integral de Minería de Datos y Aprendizaje Automático diseñada para la detección, clasificación y explicabilidad de transacciones fraudulentas en tiempo real. La arquitectura está dividida en tres módulos principales:

- **Backend API (Python 3.13 / FastAPI):** Procesa las solicitudes HTTP REST, valida entradas con Pydantic, ejecuta ingeniería de características, aplica el pipeline de Scikit-Learn (Random Forest) y persiste en PostgreSQL.
- **Machine Learning & Data Pipeline (scikit-learn / pandas):** Genera el dataset, ejecuta limpieza automatizada, extrae hallazgos exploratorios, entrena estimadores estratificados 70/30 y genera explicabilidad dinámica.
- **Frontend SPA (React 19 / TypeScript / Vite / Tailwind CSS):** Interfaz empresarial responsive con dashboard interactivo, formulario de prueba con presets, historial pericial, métricas y visualizaciones en Recharts.

2. Módulo Backend (`backend/app/`)

■ backend/app/main.py

Punto de entrada principal de la API FastAPI. Inicializa la conexión a la base de datos PostgreSQL/SQLite, registra los enrutadores REST bajo `/api`, habilita el middleware CORS para desarrollo web y gestiona el evento de precarga del modelo de Machine Learning en memoria al iniciar el servidor.

■ backend/app/config.py

Configuración global centralizada mediante Pydantic Settings. Almacena la cadena de conexión `DATABASE\_URL` (configurada para PostgreSQL `postgres://postgres:12345@localhost:5432/fraudguard\_db`), umbrales de riesgo (Bajo <40%, Medio 40-70%, Alto >=70%), recomendaciones operativas y rutas absolutas de artefactos.

■ backend/app/database.py

Gestor de persistencia relacional con SQLAlchemy. Configura el motor `create\_engine`, la fábrica de sesiones `SessionLocal` y el generador de dependencias `get\_db()`. Detecta automáticamente el dialecto de base de datos (PostgreSQL o SQLite) para adaptar los parámetros de conexión.

■ backend/app/models/db_models.py

Definición del modelo ORM `PredictionRecord` mapeado a la tabla `predictions`. Registra cada evaluación en PostgreSQL incluyendo: `prediction\_id`, `transaction\_id`, `amount`, `city`, `merchant\_category`, `fraud\_probability`, `risk\_level`, `recommendation`, `input\_data` (JSON) y `risk\_factors` (JSON).

■ backend/app/schemas/schemas.py

Esquemas Pydantic v2 para validación rigurosa de datos. `TransactionInput` valida los tipos y rangos de las 15 variables operativas. `PredictionResponse` define la estructura del resultado con probabilidad, nivel de riesgo y explicabilidad. `HistoryItemResponse` formatea los registros históricos.

■ backend/app/services/risk_service.py

Servicio centralizado de clasificación de riesgo. Recibe la probabilidad continua [0.0 - 1.0], calcula el porcentaje y asigna el nivel (BAJO, MEDIO, ALTO) junto con la recomendación operativa correspondiente según los umbrales configurados.

■ backend/app/services/prediction_service.py

Servicio de orquestación principal. Recibe la transacción validada, invoca el cálculo de variables derivadas, ejecuta ``predict_proba()``, consulta el servicio de riesgo y explicabilidad, inserta el registro en PostgreSQL vía SQLAlchemy y retorna la respuesta JSON.

■ backend/app/api/routes.py

Enrutador con todos los endpoints REST expuestos: ``/api/health``, ``/api/dashboard``, ``/api/predict``, ``/api/history``, ``/api/models``, ``/api/data-quality``, ``/api/eda``, ``/api/feature-importance`` y ``/api/train``.

3. Módulo de Machine Learning y Pipeline (``backend/ml/``)

■ backend/ml/generate_data.py

Generador de dataset sintético realista con 16,030 registros bajo la semilla 42. Modela comportamientos de gasto, horarios, distancias y patrones de fraude con ruido estocástico. Inyecta deliberadamente duplicados, nulos y montos inválidos para evaluar la etapa de limpieza.

■ backend/ml/preprocessing.py

Pipeline de depuración de datos. Elimina filas duplicadas, descarta montos $\leq \$0$, imputa ciudades y dispositivos nulos con lógica contextual, aplica normalización sintáctica y realiza capping superior en el percentil 99.9. Genera el reporte ANTES vs DESPUÉS (``data_quality.json``).

■ backend/ml/feature_engineering.py

Extractor de 6 variables derivadas con alto poder discriminante: ``monto_vs_promedio`` (ratio de gasto), ``hora_inusual`` (madrugadas 00:00-05:59), ``distancia_anomala`` (>100 km), ``ciudad_diferente``, ``intentos_fallidos_elevados`` (≥ 2) y ``dispositivo_nuevo``.

■ backend/ml/train.py

Módulo de entrenamiento. Divide los datos en 70% train / 30% test estratificado por ``is_fraud``. Construye un ``ColumnTransformer`` (StandardScaler + OneHotEncoder) ajustado únicamente sobre train para prevenir Data Leakage. Entrena Logistic Regression y Random Forest, seleccionando técnicamente este último por F1-Score y Recall.

■ backend/ml/predict.py

Motor de inferencia en tiempo real. Carga y cachea en memoria el modelo entrenado (``fraud_model.joblib``), ejecuta la ingeniería de características sobre una sola transacción y calcula ``predict_proba()``.

■ backend/ml/eda.py

Calculador automático de distribuciones multivariadas y extractor de los 5 hallazgos estadísticos demostrados con evidencia cuantitativa e interpretación pericial de negocio (``eda_findings.json``).

■ backend/ml/explain.py

Motor de explicabilidad dinámica en tiempo real. Evalúa los datos reales de la transacción contra el perfil histórico del usuario y los pesos del modelo, generando los 5 factores principales con etiqueta, impacto (ALTO/MEDIO/BAJO) y frase explicativa sin textos estáticos.

4. Módulo Frontend SPA (`frontend/src/`)

■ frontend/src/App.tsx

Componente raíz de la SPA en React. Gestiona el estado de navegación de pestañas, realiza sondeo periódico de salud del backend (`/api/health`) y renderiza la estructura con Sidebar, Header y la página activa.

■ frontend/src/services/api.ts

Cliente HTTP centralizado desarrollado con Axios. Define métodos asíncronos fuertemente tipados para comunicarse con todos los endpoints de FastAPI (`getDashboard`, `predict`, `getHistory`, `getModels`, `getDataQuality`, `getEDA`, etc.).

■ frontend/src/types/index.ts

Definiciones e interfaces TypeScript para todas las entidades del sistema: `TransactionInput`, `PredictionResponse`, `RiskFactor`, `HistoryItem`, `DashboardData`, `ModelComparisonData`, etc.

■ frontend/src/components/Sidebar.tsx

Barra lateral de navegación responsive. Contiene los enlaces a los 7 módulos del sistema y un panel inferior con el estado en tiempo real de la API, conexión PostgreSQL y modelo activo.

■ frontend/src/components/Header.tsx

Encabezado superior con títulos dinámicos según el módulo visible y botón de re-entrenamiento del pipeline en caliente con estado de carga.

■ frontend/src/components/RiskBadge.tsx

Componente de insignia visual reutilizable para niveles de riesgo BAJO (Verde), MEDIO (Amarillo) y ALTO (Rojo) con íconos vectoriales de Lucide Icons.

■ frontend/src/pages/DashboardPage.tsx

Vista principal. Muestra 4 cards de KPI superiores, gráficos de Recharts (fraude por hora, ciudad, categoría, dona de riesgo), resumen del modelo activo y la matriz de confusión visual.

■ frontend/src/pages/AnalyzePage.tsx

Formulario interactivo para análisis en tiempo real. Incluye presets de 1 clic para 'Alto Riesgo' y 'Bajo Riesgo', validador de campos, medidor de probabilidad y tarjeta visual de explicabilidad pericial.

■ frontend/src/pages/HistoryPage.tsx

Tabla de auditoría conectada a la base de datos PostgreSQL. Ofrece búsqueda en tiempo real, filtrado por nivel de riesgo, ordenamiento y modal de inspección detallada de cada predicción.

■ frontend/src/pages/ModelsPage.tsx

Página de evaluación. Presenta la tabla comparativa de métricas reales, gráficos de barras de desempeño, justificación técnica del modelo y matrices de confusión duales.

■ frontend/src/pages/DataQualityPage.tsx

Vista de calidad. Desglosa las tarjetas ANTES vs DESPUÉS, el índice de salud del dataset (99.78%) y la tabla detallada de transformaciones ejecutadas.

■ frontend/src/pages/ExplorationPage.tsx

Página de EDA. Muestra los 5 hallazgos estadísticos demostrados con tarjetas de evidencia cuantitativa e interpretación de negocio junto a gráficos de distribución.

■ frontend/src/pages/FeaturesPage.tsx

Vista de importancia de variables. Presenta el ranking gráfico de Gini Impurity de los atributos del ensamble y su justificación funcional.

5. Documentación, Pruebas y Configuración de Infraestructura

■ docs/informe_tecnico.md

Informe técnico académico completo de 16 secciones listo para la entrega oficial.

■ docs/metodologia.md

Documento formal del marco metodológico CRISP-DM adaptado a minería de datos financiera.

■ docs/hallazgos.md

Sustentación pericial de los 5 hallazgos estadísticos descubiertos en los datos reales.

■ docs/manual_funcionamiento.md

Manual explicativo de funcionamiento operativo y flujo de extremo a extremo.

■ backend/notebooks/fraud_detection_analysis.ipynb

Jupyter Notebook ejecutable de principio a fin con celdas de código, visualizaciones y conclusiones.

■ backend/test_postgres.py

Script de verificación automatizada que comprueba la conexión a PostgreSQL con usuario `postgres` y clave `12345`, creando la base de datos `fraudguard\_db` si no existe.

■ backend/test_api.py

Suite de pruebas de integración de endpoints REST con FastAPI TestClient y persistencia en base de datos.

■ docker-compose.yml & Dockerfiles

Archivos de orquestación Docker multi-stage para levantar backend FastAPI y frontend Nginx en contenedores aislados.